

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName: B. Tech		Assignment Type: Lab	AcademicYear: 2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
CourseCode	24CS002PC215	CourseTitle	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week3 - Wednesday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber: 6.3 (Present assignment number)/ 24 (Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals Lab Objectives: - To explore AI-powered auto-completion features for core Python constructs. - To analyze how AI suggests logic for class definitions, loops, and conditionals. - To evaluate the completeness and correctness of code generated by AI assistants. Lab Outcomes (LOs):	Week3 - Wednesday	

	After completing this lab, students will be able to: • Use AI tools to generate and complete class definitions and methods. • Understand and assess AI-suggested loops for iterative tasks. • Generate conditional statements through prompt-driven suggestions. • Critically evaluate AI-assisted code for correctness and clarity. Task Description#1 (Classes) • Use AI to complete a Student class with attributes and a method. • Check output • Analyze the code generated by AI tool Instructions: • Initialize class with attributes like name, roll no, marks • Method to display student details • Method to calculate grade based on marks (A: >=90, B: >=75, C: >=60, else Fail) Start Writing code and auto complete using any AI tool Expected Output#1 • Class with constructor and display_details() method PROMPT : class Student: def __init__(self, name, roll_no, marks): self.name = name self.roll_no = roll_no self.marks = marks def display_details(self): print("Student Details:") print(f"Name : {self.name}") print(f"Roll No : {self.roll_no}") print(f"Marks : {self.marks}") print(f"Grade : {self.calculate_grade()}") Now complete the remaining code and Initialize class with attributes like name, roll no, marks, Method to display student details, Method to calculate grade based on marks (A: >=90, B: >=75, C: >=60, else Fail) and display the output.	
--	--	--

```
View Go Run Terminal Help task1c.py - AI assisted coding - Cursor
task1c.py X
lab 4 > lab 6.3 > task1c.py
1 class Student:
2     def __init__(self, name, roll_no, marks):
3         self.name = name
4         self.roll_no = roll_no
5         self.marks = marks
6
7     def calculate_grade(self):
8         if self.marks >= 90:
9             return 'A'
10        elif self.marks >= 75:
11            return 'B'
12        elif self.marks >= 60:
13            return 'C'
14        else:
15            return 'Fail'
16
17    def display_details(self):
18        print("Student Details:")
19        print(f"Name      : {self.name}")
20        print(f"Roll No   : {self.roll_no}")
21        print(f"Marks     : {self.marks}")
22        print(f"Grade     : {self.calculate_grade()}")
23
```

```
View Go Run Terminal Help task1c.py - AI assisted coding - Cursor
task2.py lab test 1 task1.py lab test 1 your_file.csv task4c.py lab 4
lab 4 > lab 6.3 > task1c.py
23
24 # Example usage:
25 student1 = Student("Alice", 101, 92)
26 student2 = Student("Bob", 102, 78)
27 student3 = Student("Charlie", 103, 55)
28
29 student1.display_details()
30 print()
31 student2.display_details()
32 print()
33 student3.display_details()
34
```

```
30 print()
...
Problems Output Debug Console Terminal Ports
hon313\python.exe' 'c:\Users\kanch\cursor\extensions\ms-python.debugpy
bugpy\launcher' '49890' '--' 'c:\Users\kanch\OneDrive\Desktop\AI assist
Student Details:
Name      : Alice
Roll No   : 101
Marks     : 92
Grade     : A

Student Details:
Name      : Bob
Roll No   : 102
Marks     : 78
Grade     : B

Student Details:
Name      : Charlie
Roll No   : 103
Marks     : 55
Grade     : Fail
PS C:\Users\kanch\OneDrive\Desktop\AI assisted coding>
Ctrl+K to generate a command
```

Task Description#2 (Loops)

- Prompt AI to complete a function that prints the first 10 multiples of a number using a loop.
- Analyze the generated code
- Ask AI to generate code using other controlled looping

Write code using **For** Loop, later complete code using **While** Loop

Expected Output#2

- Correct loop-based implementation

PROMPT:

```
def print_multiples(n):
    for i in range(1, 11):
        print(n * i)
```

```
# Example
print_multiples(5)
```

Now generate the above code using while loop.

```
on View Go Run Terminal Help task2c.py - AI assisted
task1.py lab test your_file.csv task4c.py lab 4
lab 4 > lab 6.3 > task2c.py
1 def print_multiples_while(n):
2     i = 1
3     while i <= 10:
4         print(n * i)
5         i += 1
6
7 # Example
8 print_multiples_while(5)
9
```

```
8 print_multiples_while(5)
9

Problems Output Debug Console Terminal Ports
ensions\ms-python.debugpy-2025.10.0-win32-x64\bundled\libs\debugpy\lau
neDrive\Desktop\AI assisted coding\lab 4\lab 6.3\task2c.py'
5
10
15
20
25
30
35
40
45
50
PS C:\Users\kanch\OneDrive\Desktop\AI assisted coding>
```

Task Description#3 (Conditional Statements)

- Ask AI to write nested if-elif-else conditionals to classify age groups.
- Analyze the generated code
- Ask AI to generate code using other conditional statements

Table: Age Group Classification Logic

Age Range	Age Group
0 – 12 years	Child
13 – 19 years	Teen
20 – 59 years	Adult
60 years & above	Senior

Expected Output#3

- Age classification function with appropriate conditions and with explanation

PROMPT:

```
def classify_age(age):
```

```
    if age >= 0:
```

```
        if age <= 12:
```

```
            print("Child")
```

```
        elif age <= 19:
```

```
            print("Teenager")
```

```
        elif age <= 59:
```

```
            print("Adult")
```

Complete the full code using nested loop.

```
lab 4 > lab 6.3 > task3c.py
1  def classify_age(age):
2      if age >= 0:
3          if age <= 12:
4              print("Child")
5          else:
6              if age <= 19:
7                  print("Teenager")
8              else:
9                  if age <= 59:
10                     print("Adult")
11                 else:
12                     print("Senior")
13             else:
14                 print("Invalid age")
15
16 # Example usage with a nested loop to classify multiple ages
17 ages = [5, 16, 25, 65, -3]
18 for i in range(len(ages)):
19     for j in range(1): # Nested Loop (for demonstration, runs once)
20         print(f"Age: {ages[i]}")
21         classify_age(ages[i])
22         print()
23
```

```

coding'; & 'c:\Users\kanch\AppData\Local\Programs\Python\Python313\pyt
ensions\ms-python.debugpy-2025.10.0-win32-x64\bundled\libs\debugpy\laun
neDrive\Desktop\AI assisted coding\lab 4\lab 6.3\task3c.py'
Age: 5
Child

Age: 16
Teenager

Age: 25
Adult

Age: 65
Senior

Age: -3
Invalid age

PS C:\Users\kanch\OneDrive\Desktop\AI assisted coding> 

```

Task Description#4 (For and While loops)

- Generate a sum_to_n() function to calculate sum of first n numbers
- Analyze the generated code
- Get suggestions from AI with other controlled looping

Expected Output#4

- Python code with explanation

PROMPT:

```

def sum_to_n(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total

```

Example
print(sum_to_n(5)) # Output: 15

Generate the above code using while loop.

```
task4c.py lab 4 task5c.py lab 4 task1c.py
```

```
lab 4 > lab 6.3 > task4c.py
```

```
1 def sum_to_n(n):
2     total = 0
3     i = 1
4     while i <= n:
5         total += i
6         i += 1
7     return total
8
9 # Example
10 print(sum_to_n(5)) # Output: 15
11
```

[illegible]

- Use AI to build a BankAccount class with deposit, withdraw, and balance methods

- Use AI to build a Bank account class with deposit, withdraw, and balance methods.
- Analyze the generated code
- Add comments and explain code

Instructions

- Initialize BankAccount class with attributes like name, balance
- Method to deposit amount
- Method to withdraw amount

- **Method to check balance**

Expected Output#5

- Python code with explanation

PROMPT :

class BankAccount:

Constructor to initialize account holder and starting balance

def __init__(self, name, balance=0):

self.name = name

self.balance = balance

Generate the complete code and Initialize BankAccount class with attributes like name, balance, Method to deposit amount, Method to withdraw amount, Method to check balance and display the output.

```
task5c.py - AI assisted coding - Cursor
task1c.py task2c.py task3c.py task4c.py task5c.py
lab 4 > lab 6.3 > task5c.py
1 class BankAccount:
2     # Constructor to initialize account holder and starting balance
3     def __init__(self, name, balance=0):
4         self.name = name
5         self.balance = balance
6
7     # Method to deposit amount
8     def deposit(self, amount):
9         if amount > 0:
10             self.balance += amount
11             print(f"{amount} deposited. New balance: {self.balance}")
12         else:
13             print("Deposit amount must be positive.")
14
15     # Method to withdraw amount
16     def withdraw(self, amount):
17         if amount > 0:
18             if amount <= self.balance:
19                 self.balance -= amount
20                 print(f"{amount} withdrawn. New balance: {self.balance}")
21             else:
22                 print("Insufficient balance.")
23         else:
24             print("Withdrawal amount must be positive.")
25
```

```

25
26     # Method to check balance
27     def check_balance(self):
28         print(f"Current balance: {self.balance}")
29
30     # Example usage
31     account1 = BankAccount("John Doe", 1000)
32     print(f"Account Holder: {account1.name}")
33     account1.check_balance()
34     account1.deposit(500)
35     account1.withdraw(200)
36     account1.withdraw(2000)
37     account1.check_balance()
38

```

```

PS C:\Users\kanch\OneDrive\Desktop\AI assisted coding> c:; cd
coding'; & 'c:\Users\kanch\AppData\Local\Programs\Python\Pyth
ensions\ms-python.debugpy-2025.10.0-win32-x64\bundled\libs\deb
neDrive\Desktop\AI assisted coding\lab 4\lab 6.3\task5c.py'
Account Holder: John Doe
Current balance: 1000
500 deposited. New balance: 1500
200 withdrawn. New balance: 1300
Insufficient balance.
Current balance: 1300
PS C:\Users\kanch\OneDrive\Desktop\AI assisted coding>

```

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

Evaluation Criteria:

Criteria	Max Marks
Class	1.0

	Loops	1.0		
	Conditional Statements	0.5		
	Total	2.5 Marks		